

# Assignment 2

## Multi-Agent Personal Auto-Scheduling Platform

|             |            |             |                                        |
|-------------|------------|-------------|----------------------------------------|
| Total Marks | 100 Marks  | Submission  | GitHub repo + Demo video (max 10 min)  |
| Format      | Individual | Late Policy | 5% / day. No submissions after 7 days. |

### 1. Overview

Design, implement, and demonstrate a fully operational Multi-Agent Auto-Scheduling Platform running entirely on your local machine using a locally hosted LLM. A central orchestrator manages five specialized agents, handles resource scheduling, and serves a web-based dashboard.

| Layer                    | Description                                                                                                           |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Main Orchestrator        | Manages all agents; monitors CPU, RAM, disk; handles LLM scheduling and deadlock prevention; serves the dashboard.    |
| Agent-1: AI-Times        | Fetches latest AI YouTube videos (5 news + 5 personality); sends daily HTML email digest.                             |
| Agent-2: Mailman         | Monitors Gmail inbox; classifies emails with LLM; labels, stars, alerts on key people; sends daily summary.           |
| Agent-3: Wallstreet Wolf | Tracks 20+ stocks via Yahoo Finance; shows top 5 gainers/losers/watchlist; LLM commentary; daily email.               |
| Agent-4: Your Choice     | A custom agent that solves a real personal or professional problem using at least one external API and the local LLM. |

### 2. Requirements

#### 2.1 Platform & Local LLM

- Run a local LLM using Ollama or LM Studio — Model: Qwen3
- All AI inference must be local — no calls to hosted LLM APIs (OpenAI, Anthropic, etc.)
- Use SQLite, PostgreSQL, or MySQL for caching and persistent storage
- Language: Python 3.12 backend; React or plain HTML/JS frontend

#### 2.2 Main Orchestrator

- Live resource monitoring (CPU %, RAM %, disk, threads) - dashboard updates every ≤5 s
- On-screen alarm when any resource exceeds 90%; display a suggested corrective action
- LLM semaphore: only one agent calls the model at a time; no deadlocks
- Detect and restart crashed agents without restarting the full platform

#### 2.3 Agent-1: AI-Times

- YouTube Data API v3: fetch 5 AI news + 5 personality/interview videos (last 24-48 h)

## Assignment 2: Multi-Agent Personal Auto-Scheduling Platform

---

- Send daily HTML digest email on a configurable schedule
- Dashboard tab: both video sets with thumbnail, title, channel, date; manual Refresh button

### 2.4 Agent-2: Mailman

- Connect to Gmail via OAuth 2.0; scan inbox on schedule
- LLM classifies each email: Urgent / Action Required / Follow-Up / Newsletter / Notification / Personal / Other
- Apply Gmail labels and stars automatically; alert on emails from a configurable key-people list
- Dashboard tab: category breakdown, email list with AI summaries, manual scan trigger

### 2.5 Agent-3: Wallstreet Wolf

- Fetch live + historical data from Yahoo Finance for a watchlist of 20+ stocks
- Dashboard tab: Block 1 — Top 5 Gainers; Block 2 — Top 5 Losers; Block 3 — Full Watchlist
- LLM market commentary card; currency exchange pairs and precious metals (Gold & Silver) section
- Send daily market brief email on schedule

### 2.6 Agent-4: Your Choice

- At least one live external API call and one LLM processing step
- Dedicated dashboard tab with professional looking UI and user configurable parameters
- At least one automated scheduled action (email, alert, or cache refresh)
- Include a 150-word use-case proposal in your first README commit

---

## 3. Deliverables

| # | Deliverable                                          | Notes                                                                                                            |
|---|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| 1 | GitHub repository (public or shared with instructor) | <i>Only Public facing GitHub Repo will be accepted, if Repo not accessible publicly, no marks will be given.</i> |
| 2 | README.md with full setup and run instructions       | <i>Covering how to run the project from scratch.</i>                                                             |
| 3 | Architecture diagram (PDF or PNG in repo)            | <i>Show agents, orchestrator, LLM, DB, and external APIs</i>                                                     |
| 4 | Demo video — max 10 minutes                          | <i>All 5 agents running live; narrate key design decisions</i>                                                   |

## 4. Grading Rubric (Total: 100 Marks)

| # | Deliverable                           | Marks |
|---|---------------------------------------|-------|
| 1 | Orchestrator & Architecture           | 15    |
| 2 | AI-Times                              | 15    |
| 3 | Mailman                               | 15    |
| 4 | Wallstreet Wolf                       | 15    |
| 5 | Agent-4 (Your Choice)                 | 15    |
| 6 | Code Quality, Repository & Creativity | 10    |
| 7 | Demo Video                            | 15    |
|   | Total                                 | 100   |

## 5. Video Demo Instruction

- Your video should be Max 10 min; any video more than 10 min will not be accepted and will be marked 0.
- Make sure to cover only Product Demo in video, no slides, no architecture or anything else needed.
- Video should demonstrate all 5 agents working distinctively and highlight your creativity.
- Your screen size should be readable, so make sure to not to use any high-resolution monitor that makes screen size too small for audience.
- If any of the agents are not working within your demo video, full marks will be deducted for that agent from Grading Rubric above.
- Upload your video to your YouTube channel and share the link only, no other video format will be accepted.

## 6. Learning Objectives

- Learn how Agentic-AI work within multi-Agent environments.
- Building your own workable machine that can practically help to automate your day-to-day tasks.
- Learn to work with local LLM that can help you in real Product building for next Projects.
- Learn to keep track of System level resources, making sure that there are no deadlocks at process or agent level, also the resources are released after each process completion and there are no zombie processes consuming system resources.
- Apply Security wherever possible, making sure that there is no vulnerability.